SESSION 7 — Array Manipulation

Array manipulation functions are essential when:

- Combining datasets
- Splitting data for training/testing
- Modifying array structure
- Adding or removing elements
- Managing memory correctly

---

1. np.concatenate()

Why It Exists

To join arrays along an existing axis.

Syntax

np.concatenate((arr1, arr2), axis=0)

Parameters

- tuple of arrays → arrays to combine
- axis → 0 (rows), 1 (columns)

Important

Arrays must have same shape except along concatenation axis.

```
import numpy as np

a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])

print("Concatenate axis=0:\n", np.concatenate((a,b), axis=0))
print("Concatenate axis=1:\n", np.concatenate((a,b), axis=1))
```

```
Concatenate axis=0:
 [[1 2]
 [3 4]
 [5 6]
 [7 8]]
Concatenate axis=1:
 [[1 2 5 6]
 [3 4 7 8]]
```

2. Stacking Functions

Stacking adds a new dimension or stacks arrays in specific orientation.

np.vstack()

Vertical stacking (row-wise)

np.hstack()

Horizontal stacking (column-wise)

np.dstack()

Depth stacking (third dimension)

```
print("vstack:\n", np.vstack((a,b)))
print("hstack:\n", np.hstack((a,b)))
```

```
print("dstack:\n", np.dstack((a,b)))
```

```
vstack:
 [[1 2]
 [3 4]
 [5 6]
 [7 8]]
hstack:
 [[1 2 5 6]
 [3 4 7 8]]
dstack:
 [[[1 5]
  [2 6]]

 [[3 7]
  [4 8]]]
```

Stacking vs Concatenation

Concatenation:

- Joins along an existing axis.

Stacking:

- May introduce new axis (especially dstack).

Use stacking when creating higher dimensional arrays.

3. Splitting Arrays

np.split()

Splits array into equal parts.

Syntax: np.split(arr, sections, axis=0)

np.array_split()

Allows unequal splitting.

Why two functions?

np.split() requires exact division. np.array_split() handles uneven splits.

```
arr = np.arange(8)

print("Split:", np.split(arr, 4))
print("Array Split:", np.array_split(arr, 3))
```

```
Split: [array([0, 1]), array([2, 3]), array([4, 5]), array([6, 7])]
Array Split: [array([0, 1, 2]), array([3, 4, 5]), array([6, 7])]
```

4. Insert, Append, Delete

np.insert(arr, index, values, axis=None)

Inserts values before given index.

np.append(arr, values, axis=None)

Appends values at end.

np.delete(arr, index, axis=None)

Deletes elements at index.

Important: These return new arrays (original not modified).

```
arr = np.array([10,20,30])
```

```
print("Insert:", np.insert(arr, 1, 15))
print(arr)
print("Append:", np.append(arr, 40))
print(arr)
print("Delete:", np.delete(arr, 1))
print(arr)
```

```
Insert: [10 15 20 30]
[10 20 30]
Append: [10 20 30 40]
[10 20 30]
Delete: [10 30]
[10 20 30]
```

5. Copy vs View

Understanding memory behavior is critical.

View (.view())

Shares memory with original array.

Copy (.copy())

Creates independent memory.

Modifying view affects original. Modifying copy does not.

```
arr = np.array([1,2,3])

view_arr = arr.view()
copy_arr = arr.copy()

arr[0] = 100

print("Original:", arr)
print("View:", view_arr)
print("Copy:", copy_arr)
```

```
Original: [100   2   3]
View: [100   2   3]
Copy: [1 2 3]
```

```
Start coding or generate with AI.
```